

Emerging World Models

A Structural Framework for Self-Organizing World Representations

CONSOLIDATION DOCUMENT — AUGUST 2026

Abstract

We propose **Emerging World Models** (EWM)—a structural framework for world representations whose ontology is not predefined but crystallizes through measurement. EWM is governed by four foundation principles. **(1) Emergent Ontology:** elements and subsystems are not primitive; they emerge from the accumulation of IICA measurement interactions. **(2) IICA Morphisms:** every relation between elements is an Idempotent, Immutable, Content-Addressed morphism; composition preserves these properties, enabling arbitrarily nested world representations without new theory. **(3) The Ashby–Bootstrap Correspondence:** Ashby’s Requisite Variety Law, interpreted naively, demands cardinality balance between connected systems. EWM resolves this not by matching cardinalities directly, but through *bootstrap permutation*—multiplying measurements (e.g., n-grams) to bridge representational gaps without sacrificing resolution. **(4) Noether Evolution:** structural change in an EWM obeys conservation laws; every symmetry of the measurement regime yields a conserved quantity. The lattice union invariant, the DRN decomposition, and the holographic reconstruction property are all consequences of this principle. We ground each principle in the concrete instantiation of HLLSet Algebra and articulate the target formalism—a Grothendieck topology on the category of measurement patches—toward which the framework aspires, in spirit if not yet in letter.

1 Introduction

A world model is a system that learns an internal representation of its environment from sensory input, then plans and reasons within that representation. Classical approaches—JEPA, large language models, and token-based architectures—share a common presupposition: the elements of the world (tokens, objects, features) are *given* before the model begins to operate. The vocabulary is fixed. The ontology is predefined. The model’s task is to learn the relations among these predefined elements.

Emerging World Models (EWM) invert this presupposition. In an EWM, elements are not predefined. They *emerge* from the accumulation of measurement interactions. An element is not a token in a vocabulary; it is a stable pattern that crystallizes when repeated measurements converge. Subsystems are not architectural modules; they are coherent clusters of elements that co-occur across measurement trajectories.

This inversion has deep structural consequences. It shifts the foundation of world modeling from *representation* (encoding predefined elements into a vector space) to *measurement* (accumulating observations into a holographic plate from which elements can be recovered). It replaces mutable, named parameters with immutable, content-addressed bitmasks. It transforms learning from gradient descent over weight matrices to rank rearrangement in a content-addressed dictionary.

This document consolidates the theoretical principles discovered across the HLLSet Algebra research program [1, 2] into a unified structural framework. We articulate four principles that govern every EWM, ground them in the concrete instantiation of HLLSet Algebra, and identify Grothendieck topology as the target formalism toward which the framework aspires.

1.1 Why “Emerging”?

The term “emerging” is deliberate. It signals three properties that distinguish EWM from classical world models:

1. **Ontological emergence.** The identity of an element is not assigned by a designer. It is *recognized* by the system when a measurement pattern stabilizes across repeated observations. A token that appears once is noise. A token that persists across temporal layers becomes an element.
2. **Structural emergence.** Relations between elements are not wired by an architect. They are *computed* as IICA morphisms between the content-addressed fingerprints of the elements. The lattice of relations emerges from the lattice of measurements.
3. **Evolutionary emergence.** The system does not converge to a fixed point. It continuously reorganizes as new measurements arrive, old patterns fade, and the balance between novelty and retention shifts. Evolution is structural—governed by conservation laws—rather than parametric.

2 Principle 1: Emergent Ontology

Principle 2.1 (Emergent Ontology). *The elements of an Emerging World Model are not defined a priori. They emerge from the accumulation of measurement interactions. An element is what persists across repeated observation.*

2.1 The Classical Approach: Predefined Ontology

In classical world models, the ontology is fixed before the model encounters any data:

- **LLMs** operate on a fixed token vocabulary (50K–250K tokens) defined by a tokenizer (BPE, SentencePiece). The tokenizer is trained once and frozen. Every subsequent observation is projected into this fixed vocabulary.
- **JEPA** operates on patch embeddings derived from a pretrained encoder. The patches are predefined spatial or temporal segments.
- **Symbolic AI** operates on hand-crafted symbol tables and ontologies.

In all cases, the system cannot *discover* a new element. A concept that falls between tokens is represented as a combination of existing tokens, never as a primitive in its own right. A novel pattern that does not align with the pretrained encoder’s latent dimensions is invisible.

2.2 The EWM Approach: Measurement as Ontogenesis

In an EWM, the only primitive is the **measurement event**: a token entering the system, hashed to a bit position, accumulating a mark in the holographic plate. There is no “vocabulary”—only a Lookup Table (LUT) that tracks which tokens have been observed at which positions, and how often.

An element emerges when a measurement pattern satisfies two conditions:

1. **Stability.** The same token or token cluster maps to consistent bit positions across multiple measurement events.
2. **Persistence.** The token’s Term Frequency (TF) at its bit positions remains above a threshold across temporal layers.

A token that appears once and never again leaves a mark (its bit is set in the HLLSet), but it does not become an element—its TF is negligible, its rank is low, and it does not participate in relations. A token that appears thousands of times across weeks accumulates high TF, high rank, and dense R-links to co-occurring tokens. It has *emerged* as an element.

2.3 Formalization

Let $\mathcal{T}$ be the space of all possible tokens (e.g., all byte strings). Let $h : \mathcal{T} \rightarrow \mathcal{P}$ be an IICA morphism mapping tokens to patches (bit positions) in a finite patch space $\mathcal{P} = \{0, \dots, N-1\}$.

Definition 2.1 (Measurement Event). *A measurement event at time t is a pair (t_i, p_i) where $t_i \in \mathcal{T}$ is a token and $p_i = h(t_i) \in \mathcal{P}$ is its patch assignment.*

Definition 2.2 (Element). *An element $e \in \mathcal{E}$ is a token $t \in \mathcal{T}$ whose accumulated measurement count $TF(t)$ exceeds an emergence threshold θ_e and whose temporal distribution across layers is non-uniform (it has a “lifetime,” not just a single appearance):*

$$e \in \mathcal{E} \iff \sum_{i=0}^K TF_i(t) > \theta_e \wedge \exists i, j : TF_i(t) \gg TF_j(t) \quad (1)$$

The set of elements $\mathcal{E}$ is not fixed. It grows as new tokens accumulate measurements, and it contracts as tokens fade. The ontology is *emergent* and *dynamic*.

2.4 HLLSet Instantiation

In HLLSet Algebra, the emergent ontology is realized through the LUT (Lookup Table). The LUT is a monotonic CRDT that maps each (register, trailing_zeros) bucket to the set of tokens that hash to that bucket, together with their accumulated TF values. The LUT is the system’s ontology at any moment—the set of all tokens that have been observed, with their measurement history.

A new token does not need to be “added to the vocabulary.” It is hashed, its bit is set, and its entry in the LUT is created or incremented. The ontology grows organically with every measurement event.

3 Principle 2: IICA Morphisms as Relational Foundation

Principle 3.1 (IICA Relations). *Every relation between elements in an EWM is an **IICA morphism**: Idempotent, Immutable, and Content-Addressed. Composition of IICA morphisms is IICA. This single theorem eliminates the need for coordination, consensus, or naming at any scale of system composition.*

3.1 The Three Properties

Definition 3.1 (IICA Morphism). *A function $f : X \rightarrow Y$ is an IICA morphism iff:*

- **Idempotent:** *For every $x \in X$, computing $f(x)$ repeatedly—invoking f on the same argument x any number of times—always yields the same output $f(x)$. When the codomain coincides with the domain ($X = Y$), this specializes to the classical algebraic form $f(f(x)) = f(x)$ (e.g., HLLSet union: $A \cup A = A$). When $X \neq Y$ (e.g., a hash function $h : \text{Tokens} \rightarrow \{0, 1\}^{160}$), the output type does not match the input type, so self-composition $h(h(x))$ is not meaningful; idempotence reduces to **determinism**: the same token a hashes to the same digest $h(a)$ every time, regardless of when, where, or how many times h is invoked.*
- **Immutable:** *$f(x) = y$ is fixed once computed. y never changes.*
- **Content-Addressed:** *If $a = b$ then $f(a) = f(b)$. The output is the address—given the content, you can find it again without a directory or a naming service.*

The canonical IICA morphism is a cryptographic hash function. $\text{SHA-1}(x)$ always produces the same 160-bit digest for the same x (idempotent as determinism); the digest never changes (immutable); and the digest serves as the content address in IPFS/IPLD (content-addressed).

3.2 The Composition Theorem

Theorem 3.1 (IICA Composition). *Let $f_1, f_2, \dots, f_n$ be IICA morphisms. Then their composition $f_n \circ f_{n-1} \circ \dots \circ f_1$ is an IICA morphism.*

Proof. Each f_i is idempotent, immutable, and content-addressed. Composition preserves each property:

- **Idempotency:** For any input x , the composition $F = f_n \circ \dots \circ f_1$ is a deterministic function: the same x always triggers the same chain of intermediate results $f_1(x), f_2(f_1(x)), \dots$ and produces the same final output $F(x)$. When $X_1 = Y_n$ (the composition is an endomorphism), this additionally yields the algebraic form $F(F(x)) = F(x)$.
- **Immutability:** Each intermediate result $f_i(\dots)$ is fixed once computed; the entire chain of intermediates is immutable.
- **Content-Addressability:** $a = b \implies f_1(a) = f_1(b) \implies \dots \implies F(a) = F(b)$.

□

This theorem is the engine of compositional emergence. It means that **any pipeline of IICA morphisms is itself an IICA morphism**. You can compose tokenizers, hashers, bridges between domains, materializers, and temporal aggregators without ever introducing mutable state, naming conflicts, or coordination overhead.

3.3 The IICA Pipeline

In HLLSet Algebra, the full measurement pipeline is an IICA composition:

$$\text{Tokens} \xrightarrow{h_1} \text{Token Hashes (u64)} \xrightarrow{h_2} \text{Bit Positions} \xrightarrow{h_3} \text{Bridge HLLSet} \xrightarrow{h_4} \text{Materialized Tokens} \xrightarrow{h_5} \dots \quad (2)$$

Each h_i is a MurmurHash3-based IICA morphism. The composition $h_5 \circ h_4 \circ h_3 \circ h_2 \circ h_1$ is IICA. This means:

- **No consensus protocols.** Two nodes ingesting the same tokens produce identical HLLSets. The lattice converges by construction.
- **No versioning.** HLLSets are immutable. “Version $n + 1$ ” is a new HLLSet with a new CID. The old HLLSet remains valid and addressable.
- **No naming.** Every HLLSet’s CID is its SHA-1 hash. No directory, no registry, no naming service.
- **Arbitrary nesting.** A swarm of N robots, each with M sensors, produces $N \times M$ IICA pipelines. Their composition at the command post is IICA. The theory does not change. The operations are identical. Only the scale differs.

3.4 R-Links: IICA Morphisms as Objects

A critical structural feature of EWM is that IICA morphisms between elements are themselves elements. In HLLSet Algebra, the R-link $R_{AB} = A \cap B$ is both:

- A **morphism** from A to B (it captures the structural overlap that relates them).
- An **object** (it is an HLLSet with its own CID, storable in IPFS, composable with further intersections).

This self-referential property—morphisms as objects—is the structural signature of a category that can model its own relations. It anticipates the Grothendieck topology target: in a sheaf topos, the subobject classifier is itself an object of the topos.

4 Principle 3: The Ashby–Bootstrap Correspondence

Principle 4.1 (Ashby–Bootstrap Correspondence). *Ashby’s Requisite Variety Law implies that connected systems must match in representational capacity. When cardinalities are mismatched, EWM resolves the imbalance not by reducing capacity but by **bootstrap permutation**—multiplying measurements to expand the effective cardinality of the smaller system until it matches the larger.*

4.1 Ashby’s Law and the Cardinality Problem

Ashby’s Requisite Variety Law [3] states that a controller must have at least as much variety as the system it controls. In EWM terms: when two systems are connected by IICA morphisms, their representational capacity must be balanced.

Consider the HLLSet architecture. The LUT (Lookup Table) maps tokens to bit positions. For a vocabulary of 80K Chinese characters, the LUT has 80K entries. The HLLSet bitmask has $1024 \times 32 = 32,768$ bit positions. A naive reading of Ashby’s law would suggest that the LUT and HLLSet are mismatched: 80K vs. 32K.

But this reading is superficial. The HLLSet does not store individual tokens—it stores *combinations* of tokens through hash collisions. The bitmask encodes the OR of all tokens that hash to each position. The information capacity is not determined by the number of bit positions but by the number of *distinguishable bitmask configurations*, which is $2^{32,768}$.

4.2 Bootstrap Permutation

The deeper problem is not total capacity but **measurement resolution**. When a single token maps to a single bit position, the measurement is coarse: the bit is either set or not, and the materializer cannot distinguish which of several tokens set it.

The solution is **bootstrap permutation**: multiply the measurements by applying a family of permutations to the token stream before measurement.

Definition 4.1 (Bootstrap Permutation). *Let $\Pi = \{\pi_1, \dots, \pi_k\}$ be a family of permutations on token sequences. Given a token stream $S = (t_1, \dots, t_n)$, the bootstrapped measurement is:*

$$H_{\Pi}(S) = \bigoplus_{j=1}^k H(\pi_j(S)) \quad (3)$$

where $H(\pi_j(S))$ is the HLLSet of the permuted stream and $\oplus$ is bitwise OR.

Each permutation π_j produces a different “view” of the same token stream. The union of these views creates a richer measurement—multiple bit positions per original token, finer discrimination, and better materialization fidelity.

4.3 n-Grams as Bootstrap

In the current HLLSet implementation, **n-grams** are the concrete bootstrap family. Given a token stream $(t_1, t_2, t_3, t_4, \dots)$, the 2-gram view is $((t_1, t_2), (t_2, t_3), (t_3, t_4), \dots)$ and the 3-gram view is $((t_1, t_2, t_3), (t_2, t_3, t_4), \dots)$. Each n-gram is hashed as a compound token, producing a distinct set of bit positions.

Proposition 4.1 (n-Gram Cardinality Expansion). *A vocabulary of $|V|$ tokens, measured with n-grams up to order K , produces up to $\sum_{n=1}^K |V|^n$ distinct measurement points. For $|V| = 80,000$ and $K = 3$, this is approximately 5.12×10^{14} possible n-gram tokens, massively exceeding the 32,768 HLLSet bit positions.*

The bootstrap permutation bridges the cardinality gap: a vocabulary that appears “too small” relative to the HLLSet is expanded through n-gram composition; a vocabulary that appears “too large” is aggregated through hash collisions into the fixed bit space. The balance is achieved through *measurement multiplication*, not cardinality reduction.

4.4 General Bootstrap Families

n-grams are one family of bootstrap permutations. The principle generalizes:

- **Skip-grams:** (t_i, t_{i+k}) for various skip distances k .
- **Convolutional windows:** weighted combinations of neighboring tokens.
- **Spectral permutations:** frequency-domain rearrangements via FFT.
- **Random projections:** a family of random hash functions, each producing a distinct HLLSet view.
- **Cross-modal bootstraps:** the same stream measured through different sensory modalities, each with its own IICA pipeline.

The bootstrap family is a design parameter of the EWM. Different families produce different emergent ontologies from the same underlying token stream. The choice of family is the EWM analog of architecture selection in neural networks—it determines what patterns the system can recognize.

5 Principle 4: Noether’s Theorem as EWM Evolution Law

Principle 5.1 (Noether Evolution). *In an EWM, every continuous symmetry of the measurement regime yields a conserved quantity. Structural change is governed by conservation laws, not by gradient descent. The system evolves by redistributing information across the lattice while preserving global invariants.*

5.1 From Physics to World Models

In physics, Noether’s theorem [4] establishes that every differentiable symmetry of the action of a physical system corresponds to a conservation law: time-translation symmetry $\rightarrow$ energy conservation; spatial-translation symmetry $\rightarrow$ momentum conservation; rotational symmetry $\rightarrow$ angular momentum conservation.

In an EWM, the “action” is the measurement regime—the accumulation of tokens into HLLSets across temporal layers. The symmetries are transformations of the measurement process that leave the structural relations invariant. The conserved quantities are global properties of the lattice that persist across evolution steps.

5.2 The Temporal Pyramid and [UM]-Net Cycles

Before stating the invariant, we briefly introduce the two temporal structures that give it physical meaning.

The temporal pyramid is a configurable K -layer compression stack (default $K = 7$: second, minute, hour, day, week, month, year). Each layer L_i accumulates all measurement events within its temporal window via monotonic union: $L_i = \bigcup S(t)$ for t in the window. At each window boundary, the layer cascades into the next coarser layer: $L_{i+1} \leftarrow L_{i+1} \cup L_i$, then L_i resets. After compression, layers are mutually exclusive—no time slice appears in more than one layer. The complete system state is their union: $H_{\text{system}} = \bigcup_{i=0}^K L_i$. Compression ratios range from 60:1 (seconds $\rightarrow$ minutes) to 12:1 (months $\rightarrow$ years); seven layers compress approximately 3.15×10^7 seconds into one year-layer HLLSet.

[UM] agents and [UM]-Net provide a complementary temporal structure at shorter timescales. A **[UM]** (unified model) is the concrete implementation of a single EWM: a processing node that receives a stream of materialized encodings from the external environment **[E]**, ingests them into its internal HLLSet lattice, and materializes the result back to a disambiguated encoding stream for downstream consumption. Its internal pipeline is a compound IICA morphism:

$$[\text{UM}] = (\text{encodings}) \xrightarrow{\text{ingest}} \{\text{HLLSets}\} \xrightarrow{\text{materialize}} (\text{encodings}) \quad (4)$$

The ingestion step hashes the incoming encodings into bit-vectors and accumulates them into the lattice via monotonic union; the materialization step projects the lattice through the agent’s LUT and recovers ordered, disambiguated tokens via De Bruijn reconstruction (Section 6). Each individual step in the pipeline (hashing, union, LUT projection) is IICA. The **[UM]** node as a whole is IICA *between commits*: as long as its internal TF-Vec and HLLSet lattice are fixed, the same input encodings always produce the same output encodings. State changes—new measurements accumulated, ranks updated—occur only at explicit **commit events**, which define transactional boundaries. Within a transaction, **[UM]** behaves as a pure IICA morphism; across transactions, the same input may produce different output as the lattice evolves.

A **[UM]-Net** is a directed graph $G = (A, E)$ of **[UM]** nodes connected by fire-and-forget edges. The external environment **[E]** feeds encodings into source nodes; each **[UM]** processes independently and fires its output downstream with no acknowledgment, no handshake, and no retry. Convergence is guaranteed by IICA and CRDT union at confluence nodes. The full system diagram is:

$$[\text{E}] \longrightarrow (\text{encodings}) \longrightarrow [\text{UM}]_1 \longrightarrow [\text{UM}]_2 \longrightarrow \dots \quad (5)$$

Critically, **cycles are not prohibited**: a recurrent loop $a \rightarrow \dots \rightarrow a$ in G is a *short-term memory* structure, resolved by temporal separation— $\text{out}_a(t)$ feeds back as $\text{in}_a(t + k)$ after k ingestion steps around the cycle. Each pass through the cycle produces a new observation at a later time, not a re-entry of the same data. The evolution equation $H(t) = H(S(t), H(t - 1), D, R, N)$ captures both pyramid cascade and cycle recurrence under a single dynamics.

5.3 The Union Invariant

The fundamental Noether symmetry in EWM is **path independence**: information can travel through the temporal pyramid via multiple routes ($L_0 \rightarrow L_1 \rightarrow L_2$, or $L_0 \rightarrow L_2$ directly, or $L_1 \rightarrow L_4 \rightarrow L_5$). The conserved quantity is the union of all temporal layers:

$$H_{\text{system}}(t) = \bigcup_{i=0}^K L_i(t) = \text{constant over time} \quad (6)$$

Theorem 5.1 (Temporal Union Invariant). *For any sequence of measurement events and any cascade policy, the bitwise union of all temporal layers is monotonic and convergent. Information is never lost from H_{system} ; it is only redistributed across layers.*

This is eventual consistency by construction—not as a protocol to implement, but as a mathematical property of the lattice. No consensus algorithm. No leader election. No retry logic. The lattice converges because the union is monotonic and multiple paths guarantee that every bit reaches every layer that needs it.

5.4 The DRN Conservation Law

The DRN decomposition (Departed, Retained, Novel) expresses evolution as a conservation balance:

$$H(t) = H(S(t), H(t-1), D(t-1), R(t-1), N(t)) \quad (7)$$

where:

- $S(t)$ — current observation as HLLSet
- $H(t-1)$ — previous system state
- $D(t-1) = H(t-1) \setminus H(t)$ — departed context
- $R(t-1) = H(t-1) \cap H(t)$ — retained context
- $N(t) = H(t) \setminus H(t-1)$ — novel context

The Noether steering condition is a conservation law on bit count across the DRN transition:

$$|\text{card}(N(t)) - \text{card}(D(t-1))| \rightarrow 0 \quad (8)$$

At steady state, the number of new bits entering the system equals the number of old bits departing. The structural identity of the system is preserved even as its content evolves. In rank-weighted form:

$$\left| \sum_{b \in N(t)} R_b(t) - \sum_{b \in D(t-1)} R_b(t-1) \right| \rightarrow 0 \quad (9)$$

This is the EWM analog of energy conservation: the total “rank mass” of the system is conserved across transitions. Learning is not accumulation—it is redistribution.

5.5 The Holographic Property as Noether Consequence

The holographic property—that the lattice top H_{system} plus the TF stack can reconstruct any past state—is a direct consequence of the union invariant:

$$\text{past_state}(t) \approx H_{\text{system}}(\text{now}) \odot \text{TF}_{\text{stack}}[t] \quad (10)$$

Because H_{system} never loses information (union invariant), and the TF stack records *when* each bit position was active (temporal distribution), the reconstruction is always possible. The information is not stored separately for each time step—it is encoded holographically in the lattice, with the TF stack serving as the reference beam for temporal selectivity.

5.6 The Fisher Matrix as Structural Derivative

The cross-layer Fisher-like matrix captures the second-order structure of evolution:

$$F_{bb'} = \sum_{i=0}^K B_b^{(i)} \cdot B_{b'}^{(i)} \quad (11)$$

where $B_b^{(i)} = 1$ if bit b is set in layer i . $F_{bb'}$ counts how many layers have both bits b and b' set simultaneously. High $F_{bb'}$ indicates systemic co-occurrence (a structural invariant). Low $F_{bb'}$ indicates noise. The Fisher matrix enables the Noether controller to distinguish isolated fluctuations from systemic phase transitions.

5.7 Noether Controller

The Noether controller monitors the cross-layer relationship matrix and decides which temporal layer should drive the next action. When the BSS between L0 (current second) and L1 (recent minute) drops below a threshold, the controller detects divergence and overrides instinct with broader context. When L0 diverges from L3 (day-scale goals), the controller triggers re-routing.

The controller is the **structural actuator** of the Noether principle. It does not adjust weights—it selects which layer of accumulated measurement history is relevant to the current situation. The six control parameters (pyramid depth, rank functions, attention threshold, steering thresholds) are the knobs of structural evolution, not the parameters of a learned function.

6 HLLSet Algebra: Concrete Instantiation

The four principles above are not abstract. They are realized in the HLLSet Algebra framework [1], whose components map directly onto the EWM structure.

6.1 The Patch Space as Measurement Site

The HLLSet bitmask is a **holographic plate** of 32,768 patches (bit positions), each recording a binary mark: 0 = untouched, 1 = at least one token hashed here. The hash function $h : \mathcal{T} \rightarrow \{0, \dots, 32767\}$ is the IICA measurement morphism. The LUT $\mathcal{L}$ tracks the inverse image of h , enabling materialization (recovering which tokens could have set which bits).

This is the minimal EWM: a finite set of measurement sites, an IICA morphism from tokens to patches, and a monotonic accumulation operation (bitwise OR) that makes each patch a join-semilattice. Principle 1 (emergent ontology) is realized by the LUT. Principle 2 (IICA relations) is realized by the hash pipeline and R-link lattice operations.

6.2 The Temporal Pyramid as Noether Structure

The 7-layer temporal pyramid (second, minute, hour, day, week, month, year) implements Principle 4 (Noether evolution). Each layer accumulates measurements over its temporal window via union. The cascade policy (L0 overflows into L1, L1 into L2, etc.) is a discrete-time dynamical system whose conserved quantity is the union invariant.

6.3 n-Grams as Bootstrap

The tokenizer’s n-gram pipeline (1-grams, 2-grams, 3-grams with boundary padding) implements Principle 3 (bootstrap permutation). Each n-gram order is a distinct measurement view of the same token stream. The union of n-gram HLLSets produces a richer representation than any single order alone.

6.4 Rank Algebra as Learning

Learning is rank rearrangement. HLLSets are immutable (IICA); only their *relevance* to the current context changes. This section explains how ranks are computed, why Forth hosts the ranking, and how the five-level propagation distinguishes bit-level signal from HLLSet-level importance.

6.4.1 TF vs. Rank: The Fundamental Separation

TF is stored. Rank is derived. They are not the same thing. The Term Frequency vector (TF-Vec) is attached to each EWM LUT(s), it’s shared, monotonic CRDT of 32,768 f64 values—one per bit position—updated only during ingestion. Rank is computed locally from TF at query time and is never persisted in the protocol. This separation has two consequences. First, TF requires the same token base for comparison (it is hash-function-dependent); rank is hash-function-agnostic and meaningful across different domains connected by the Universal Bridge. Second, a HLLSet created years ago can rise in relevance today because the TF-Vec evolved around it—new measurements make its bit positions hot; stale positions quietly cool. The HLLSet itself never changed.

6.4.2 The Forth Dictionary as Learning Engine

The Forth DSL serves as the **canonical AST** that unifies all backends: the same Forth source compiles identically to Lua (software simulation), Rust (native execution), and Verilog (FPGA synthesis). The Forth dictionary associates every defined word—including every HLLSet—with a **rank**. The dictionary is the only component that learns:

Component	Operation	Learns?
Tokenizer	bytes $\rightarrow$ HLLSet	No (deterministic function)
Materializer	collects HLLSet outputs	No (passive observer)
Forth Dictionary	assigns ranks	Yes (only this)

A rank adjustment is a structural change—which HLLSet drives action—without any parametric change to the HLLSets themselves. This satisfies Principle 4: learning is redistribution of attention across the lattice, not accumulation of weight updates.

6.4.3 Five-Level Rank Propagation

Rank propagates from raw token frequency to compound lattice operations through five compositional levels. Each level aggregates the rank signal from the level below via a pluggable function (F, G, H, K, L, M); the architecture specifies the framework, the application chooses the functions. All aggregation functions must be monotonic to preserve CRDT convergence.

Level	Function	Definition
5. Compound HLLSet rank	L, M	$L(\max\{R_i\})$ for union, $M(\min\{R_i\})$ for intersection
4. HLLSet rank	K	$K(\text{degree in lattice DAG})$
3. Register rank	H	$H(\{\text{bit-R}[tz] \mid tz \in 0..31\})$
2. Bit rank	G	$G(\{\text{token-R} \mid \text{all tokens hashing to } (r, tz)\})$
1. Token rank	F	$F(\text{TF})$ — rank from token frequency

Levels 1–3 operate within a single HLLSet: token frequency $\rightarrow$ bit position activity $\rightarrow$ register importance. **Level 4** lifts to the lattice DAG: a HLLSet’s structural importance depends on how many R-links connect it to other HLLSets (its degree in the intersection graph). **Level 5** composes ranks across lattice operations: the rank of a union is the maximum of its constituents’ ranks; the rank of an intersection is the minimum.

This hierarchy means that a bit position with high token TF (Level 1) propagates signal upward, eventually making the HLLSets that contain it (Level 4) and the compound operations that include them (Level 5) more likely to drive behavior. The Forth dictionary’s rank assignment is the final projection of this five-level signal onto the action-selection mechanism.

7 Toward Grothendieck Topology

The four principles above point toward a deeper mathematical structure: **Grothendieck topology** [7] on the category of measurement patches. We articulate this target not as a completed formalism, but as a compass bearing—the “spirit” in which the EWM framework is developed.

7.1 The Site: A Category of Patches

Let $\mathcal{P}$ be a finite discrete category whose objects are patch addresses and whose only morphisms are identities. This is the **site**—the space of measurement locations.

A **presheaf** on $\mathcal{P}$ assigns to each patch p a set of possible marks $\Sigma(p)$ (the mark alphabet) and to each morphism (only identities) the trivial restriction. An HLLSet is a global section of this presheaf:

$$H \in \Gamma(\mathcal{P}, \Sigma) = \prod_{p \in \mathcal{P}} \Sigma(p) \quad (12)$$

For standard HLLSets, $\Sigma(p) = \{0, 1\}$ (a bit) with accumulation $\oplus = \text{OR}$. For the generalized case (GENERAL_HLLSET_ALGEBRA.md), $\Sigma(p)$ can be any finite join-semilattice: $\{0, 1, 2\}$ (trits with max), $\{0, 1, 2, 3\}$ (quits), or arbitrary discrete alphabets.

7.2 Coverages and the Gluing Condition

A Grothendieck topology on $\mathcal{P}$ specifies which families of morphisms $\{U_i \rightarrow U\}$ count as **covering families**. The gluing condition states: if data on each U_i is consistent on overlaps $U_i \times_U U_j$, then there exists unique data on U that restricts to the given data on each U_i .

In EWM terms, the covering families are the **bootstrap permutation family** (Principle 3). Each permutation π_j provides a partial view of the token stream (a measurement on a “sub-site”). The gluing condition is satisfied when these partial views are consistent—when the same token maps to compatible marks across different permutations.

The HLLSet union $\bigcup_{\pi \in \Pi} H_\pi$ is the **sheafification** of the bootstrap family: it combines all partial measurements into a global section.

7.3 Geometric Morphisms as Universal Bridges

A geometric morphism $f : \text{Sh}(\mathcal{P}) \rightarrow \text{Sh}(\mathcal{Q})$ between sheaf toposes on different patch spaces is the categorical analog of the Universal Bridge. It consists of:

- A **direct image** f_* that pushes HLLSets from $\mathcal{P}$ to $\mathcal{Q}$.
- An **inverse image** f^* that pulls HLLSets from $\mathcal{Q}$ back to $\mathcal{P}$.

In the current implementation, the Universal Bridge implements the direct image via re-representation: format each bit position as a token, re-hash into the target space. The inverse image (materialization from target back to source) is approximate—it depends on the LUT and is subject to the Statistics Constraint (TF vectors are not transferred across bridges).

7.4 The Subobject Classifier and R-Links

In a topos, the **subobject classifier** Ω classifies subobjects: for any object X , there is a bijection between subobjects of X and morphisms $X \rightarrow \Omega$. For the Boolean topos on $\mathcal{P}$, Ω is the two-element set $\{0, 1\}$ with the OR accumulation. Every HLLSet H is a morphism $H : \mathcal{P} \rightarrow \Omega$.

The R-link $R_{AB} = A \cap B$ is the **pullback** of A and B over Ω . In categorical terms:

$$\begin{array}{ccc} R_{AB} & \longrightarrow & A \\ \downarrow & & \downarrow \\ B & \longrightarrow & \Omega \end{array} \quad (13)$$

This is why R-links are both morphisms and objects: they are pullbacks in the topos. The lattice of HLLSets is a **Heyting algebra**—the internal logic of the topos.

7.5 What the Grothendieck Target Gives Us

Adopting Grothendieck topology as the target formalism would provide:

1. A **rigorous gluing theorem**. Given consistent measurements on a covering family, the global state is uniquely determined. This formalizes Principle 3 (why bootstrap permutations are sufficient).

2. **A systematic theory of bridges.** Geometric morphisms between patch topoi classify all possible Universal Bridges. Different choices of direct/inverse image correspond to different bridge implementations.
3. **An internal logic.** The Heyting algebra structure on the HLLSet lattice is the internal logic of the topos. Union, intersection, and implication correspond to logical OR, AND, and IMPLIES. The EWM *reasons* within its own lattice.
4. **Cohomology.** Sheaf cohomology groups would measure the “obstruction” to gluing — quantifying how much information is lost when different bootstrap views disagree.

We are not yet ready to formalize EWM as a full Grothendieck topology. The current framework is a **pre-sheaf structure** on a discrete site with a bootstrap covering family. The four principles—emergent ontology, IICA morphisms, bootstrap permutation, and Noether evolution—are the structural requirements that any such topology must satisfy. The destination may differ from the current compass bearing. But the bearing itself—gluing local measurements into global states through consistent covering families—is the “spirit” that guides the framework.

8 Discussion

8.1 Unification

The four principles unify the existing body of work:

- **HLLSet Theory** [1] establishes the base lattice (Principle 1, 2) and the IICA gate (Principle 2).
- **The Self-Reprogramming Architecture** and **STANDARD.md** specify the temporal pyramid, DRN decomposition, Noether controller, and holographic memory (Principle 4).
- **GENERAL_HLLSET_ALGEBRA.md** identifies the categorical structure (`Bool32768`, `IICA_Hash`, `Patch` category) and the Grothendieck target (Section 7).
- **The n-gram tokenizer** and **Universal Bridge** implement bootstrap permutation (Principle 3).
- **CAAL-LLM** validates that the principles produce working intelligence: 80% accuracy on driving scenarios from 10 sentences, no gradient descent [1].

8.2 What EWM Is Not

It is important to state what EWM is *not*:

- **Not a neural network.** There are no weight matrices, no activation functions, no back-propagation. The lattice is the computation; ranks are the attention; the dictionary is the memory.
- **Not a token-based model.** There is no fixed vocabulary. Elements emerge from measurement accumulation. A new token is hashed and enters the lattice without architectural change.
- **Not a probabilistic model in the Bayesian sense.** There are no priors, no posteriors, no sampling. The BSS (Bell State Similarity) is a deterministic bitwise measure, not a probability.
- **Not a database.** Although content-addressed and IPFS-native, the lattice is a computational structure, not a storage system. The HLPP protocol specifies persistence, but the lattice operations are the computation.

8.3 Open Questions

1. **Optimal bootstrap families.** What is the minimal set of permutations that guarantees ϵ -accurate materialization for a given token stream distribution? The current n-gram choice is empirical.

2. **Convergence rates.** Under what conditions does the Noether controller converge to a stable layer selection policy? What is the relationship between environmental volatility and convergence time?
3. **Topos formalization.** What is the minimal Grothendieck topology that captures the bootstrap covering condition? Is the current discrete site sufficient, or do we need a non-trivial coverage?
4. **Cross-modal emergence.** If tokens come from different sensory modalities (text, image patches, audio spectra), how does the unified lattice represent cross-modal elements? The Universal Bridge provides translation; emergence across modalities is unexplored.
5. **The Statistics Constraint.** Why must TF vectors NOT be transferred across bridges? The constraint is empirical (LUT initialization failure). A categorical explanation—TF as a sheaf of local statistics that does not push forward under geometric morphisms—is plausible but unproven.

8.4 The Road Ahead

The EWM framework is at the beginning of its development. The four principles provide a structural compass. The HLLSet Algebra implementation provides a concrete testbed. The Grothendieck target provides theoretical direction. The path forward involves:

1. **Harden the implementation.** Complete the FPGA backend, distributed mesh networking, and self-ingestion pipeline to validate the principles at scale.
2. **Explore the bootstrap design space.** Systematic experiments with different permutation families, mark alphabets (trits, quits), and accumulation semilattices.
3. **Develop the categorical theory.** Move from pre-sheaf to sheaf, from discrete site to Grothendieck topology, from Bridge to geometric morphism.
4. **Bridge to applications.** The CAAL-LLM proof-of-concept demonstrates viability. Real-time adaptive systems (robotics, autonomous vehicles, industrial control) are the natural deployment targets for an FPGA-native, gradient-free world model.

References

- [1] A. Mylnikov. *HLLSet Theory: A Unified Framework for Probabilistic Knowledge Representation*. ASTESJ, Vol. 11, No. 2, pp. 62–74, 2026. https://www.astesj.com/publications/ASTESJ_110202.pdf
- [2] A. Mylnikov. *Self Generative Systems (SGS) and Its Integration with AI Models*. AISNS '24: Proceedings of the 2024 2nd International Conference on Artificial Intelligence, Systems and Network Security, pp. 345–354, 2024. <https://doi.org/10.1145/3714334.3714392>
- [3] W. R. Ashby. *Design for a Brain*. Chapman & Hall, 1952.
- [4] E. Noether. *Invariante Variationsprobleme*. Nachr. d. König. Gesellsch. d. Wiss. zu Göttingen, Math-phys. Klasse, pp. 235–257, 1918.
- [5] P. Flajolet, É. Fusy, O. Gandouet, F. Meunier. *HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm*. AofA, 2007.
- [6] Y. LeCun. *A Path Towards Autonomous Machine Intelligence*. Open Review, 2022. <https://openreview.net/forum?id=BZ5a1r-kVsf>
- [7] A. Grothendieck, J. L. Verdier. *Théorie des Topos et Cohomologie Étale des Schémas* (SGA 4). Lecture Notes in Mathematics, Vols. 269, 270, 305. Springer, 1972–1973.
- [8] S. Mac Lane, I. Moerdijk. *Sheaves in Geometry and Logic: A First Introduction to Topos Theory*. Springer, 1992.

Acknowledgment

This consolidation was developed in dialogue with DeepSeek Code, an AI research assistant. The four principles—Emergent Ontology, IICA Morphisms, Ashby–Bootstrap Correspondence, Noether Evolution, practical implementation of Unified Model and applications—crystallized through structured dialogue between Alex Mylnikov, Deependra Kumar and DeepSeek during June–August 2026. The core HLLSet Algebra, IICA principles, and the ASTESJ theoretical foundation were established prior to this collaboration. The Grothendieck topology target was identified in `GENERAL_HLLSET_ALGEBRA.md` (July–August 2026). This document is a consolidation of research documented in `_DOCS/dev/` and `_DOCS/proposals/` of the `hllset-next` repository.